

Problem A. Traveling in Cells 2

The first observation is a closure operation on obstacles. If two obstacles have the form $(x, y + 1)$ and $(x + 1, y)$, then the cells (x, y) and $(x + 1, y + 1)$ may also be filled as obstacles. Repeat this operation until no such pair remains. After this closure, every connected component has a staircase-like shape: it occupies several rows, each occupied row is an interval, and both the left and right endpoints of these intervals are monotone nondecreasing from top to bottom.

For a query rectangle, construct its true lower boundary as follows. Start from the upper-left corner. At every step, go down whenever possible; continue until reaching either the bottom boundary or the right boundary. If the path reaches the right boundary, the answer is immediately 0. Call this path P . Next, start from the lower-right corner and go left whenever possible until reaching the left boundary; reaching the upper boundary means that the required path does not exist. Call this path Q . The path formed by following P to the intersection point of P and Q , and then following Q to the lower-right corner, is the true lower boundary.

The true upper boundary is obtained by the symmetric construction. The answer is the area enclosed by the two true boundaries, minus the total area of closed obstacle components enclosed by those boundaries.

The remaining difficulty is to support many queries. The four greedy walking rules naturally form four trees. After building these trees, the enclosed areas can be accumulated by a difference technique, and boundary traversal can be handled by binary lifting.

Problem B. Exchanging Kubic 3

Let b be the prefix sum array of a . One operation is equivalent to choosing an index $i \in [0, n)$ with $b_i < b_{i+1}$, and then replacing either b_i by b_{i+1} or b_{i+1} by b_i . The goal is to make b nondecreasing.

Let the final sequence be $c_0, c_1, \dots, c_n$. The sequence c can be partitioned into intervals of equal value. For each interval $[l, r]$, there is an index $x \in [l, r]$ such that $b_l \leq b_x \leq b_r$, and all values $c_l, \dots, c_r$ are obtained by expanding the value b_x . For fixed l, r, x , the cost is

$$r - l + \sum_{i=l}^x [b_i > b_x] + \sum_{i=x}^r [b_i < b_x].$$

A direct dynamic programming over intervals gives an $O(n^2)$ or $O(n^3)$ solution.

To optimize it, fix the key elements

$$0 = x_0 \leq x_1 \leq \dots \leq x_m = n$$

that appear during the transition process. For two consecutive key elements x_k, x_{k+1} , we require $b_{x_k} \leq b_{x_{k+1}}$, and their contribution is

$$\min_{y_k} \left(x_{k+1} - x_k - 1 + \sum_{i=x_k}^{y_k} [b_i < b_{x_k}] + \sum_{i=y_k+1}^{x_{k+1}} [b_i > b_{x_{k+1}}] \right).$$

By an exchange argument, there exists an optimal solution such that all intermediate values $b_{x_k+1}, \dots, b_{x_{k+1}-1}$ are outside the interval $[b_{x_k}, b_{x_{k+1}}]$.

Define $f_{i,j,t}$ as the best answer after considering $b_1, \dots, b_i$, where the last key element is j , and $t \in \{0, 1\}$ indicates whether the current position is past y_k . The transitions can be represented by small matrices. Maintaining these matrix products with lazy propagation on a segment tree gives an $O(n \log n)$ solution.

Problem C. Longest Increasing Subsequence

Let b_1 be the sequence b sorted in decreasing order, and let $A = \text{LIS}(a)$. Then

$$\text{LIS}(a + b_1), \text{LIS}(b_1 + a) \in \{A, A + 1\}.$$

If these two values are equal, then b_1 itself is a valid answer.

Otherwise, by symmetry assume

$$\text{LIS}(a + b_1) = A + 1, \quad \text{LIS}(b_1 + a) = A.$$

Because the decreasing order minimizes $\text{LIS}(a+b)$ over all permutations of b , every permutation satisfies $\text{LIS}(a+b) \geq A+1$. On the other hand, the increasing order maximizes $\text{LIS}(b+a)$. If even that maximum is at most A , then no solution exists.

Now construct a permutation b such that both concatenations have LIS equal to $A+1$. Starting from b_1 , reverse the shortest suffix that makes $\text{LIS}(b+a) = A+1$. Call the resulting sequence b_2 . If

$$\text{LIS}(a+b_2) = A+1 = \text{LIS}(b_2+a),$$

then b_2 is a valid answer.

If instead $\text{LIS}(a+b_2) > A+1 = \text{LIS}(b_2+a)$, then there is no solution. Indeed, the LIS of $a+b_2$ must use the suffix that was reversed. Suppose another sequence b_3 were valid, and let $\text{LIS}(b_3) = \text{LIS}(b_2) + k$. Then the structure of the LIS gives

$$\text{LIS}(a+b_3) = \text{LIS}(a+b_2) + k, \quad \text{LIS}(b_3+a) \leq \text{LIS}(b_2+a) + k,$$

which preserves the strict gap and contradicts validity.

Problem D. Puzzle: Nurikabe

A set of k white cells can cover at most $2k+2$ internal lattice points. Hence a necessary lower bound is

$$2k+2 \geq (n-1)(m-1).$$

This bound is achievable whenever the clue cell is not on the boundary. If the clue is on a boundary but not in a corner, the first step necessarily wastes two internal points, which corresponds to losing one unit from the ideal construction. If the clue is in a corner, the first two steps waste four internal points, corresponding to losing two units.

The implementation is therefore a constructive case analysis around this bound. The delicate cases are $\min(n, m) = 1$, $\min(n, m) = 2$, and the parity case where both dimensions are even. These cases should be handled separately before applying the general construction.

Problem E. Quad Kingdoms Chess 2

We count the number of possible piece orders leading to each result, and divide by the total number of orders to obtain the three probabilities modulo 998244353.

First consider the case with no bombs. If the maximum value in a differs from the maximum value in b , the side with the larger maximum value wins. If the maxima are equal, compare the number of occurrences of the maximum. If these counts differ, the side with more maximum pieces wins. If both counts are equal, the last maximum pieces must fight and disappear together, so the game reduces to the suffixes after those last maximum positions.

This recursive rule is captured by the non-strict suffix maximum sequence. For example,

$$f(5, 3, 2, 4, 5, 3, 4, 2, 1) = (5, 5, 4, 2, 1).$$

Compare $f(a)$ and $f(b)$ lexicographically: if $f(a) > f(b)$ then a wins, if $f(a) < f(b)$ then b wins, and if they are equal the result is a draw.

This can be counted by DP over values. Let $F_{v,0}$ and $F_{v,1}$ be the number of arrangements after placing all values at least v , where the current result is respectively “ b wins” and “draw”. The number of arrangements where a wins is obtained by subtraction. From a draw state, to remain a draw we append the same number d of value- v pieces to the end of both sequences, then distribute all remaining value- v pieces into earlier gaps by stars and bars. From a draw state to a b -winning state, we append at least one extra value- v piece to b ’s end. From a b -winning state, inserting value- v pieces cannot change the winner, so only the distribution count is needed.

Now add k bombs. A bomb splits the sequence into segments. For a segment ending in a bomb, let the parts before the bomb be A and B , and let the piece eliminated by the bomb be C . Such a segment is valid if either

$$f(A) = f(B),$$

or

$$f(A) > f(B) \quad \text{and} \quad f(A) < f(B + C).$$

The second case can also be integrated into the value DP.

Use states

$$F_{v,x,y,z}, \quad z \in \{0, 1\},$$

after all values at least v have been processed. Here x is the number of bombs whose matching piece has not yet been fixed, y is the number of already matched bomb segments whose prefix parts are still draws, and $z = 1$ means that the final segment is still a draw, while $z = 0$ means that b is already winning there.

When processing value v , choose:

- a of the x unmatched bombs to be matched with value v ;
- b of the y draw bomb-prefix segments where an extra v is appended on the a -side, making $f(A) > f(B)$;
- if $z = 1$, optionally choose $c = 1$ to append an extra v to the final b -side, making b win;
- a total number d of paired (v, v) additions, distributed among the currently draw segments.

The choice contributes binomial factors such as $\binom{x}{a} \binom{y}{b}$, and the remaining equal values are distributed into legal gaps by stars and bars. The transition goes to

$$F_{v,x-a,y-b+a,z-c}.$$

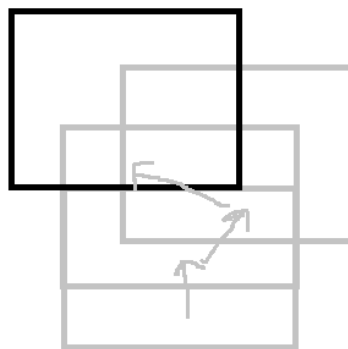
The initial state is $F_{\infty,k,0,1} = 1$. The draw count is

$$\sum_i F_{1,0,i,1},$$

and the b -win count is computed similarly; the a -win count is the total minus these two. The time complexity is $O(nk^4)$.

Problem F. World of Rains

It is hard to follow individual raindrops, because their positions are uncertain. Instead, use a relative movement viewpoint: keep the drops fixed and move the rectangular grid region.



If a lattice point is contained in K rectangles during its lifetime, then it has exactly $K + 1$ choices: there may be no raindrop there, or the raindrop may have been placed during one of the K containing rectangles. Therefore we multiply $(K + 1)$ over all lattice points. Whenever a moved rectangle no longer contains a lattice point, immediately multiply its contribution into the answer and reset its counter, because a raindrop blown out of the rectangle will never return.

Now optimize this process column by column. If a column is not contained in the rectangle at some time, then all counters in that column are reset. Hence it is enough to compute the contribution of each maximal time interval in which the column is continuously covered.

Suppose a column is continuously covered for k seconds. From bottom to top, the coverage counts are

- $1, 2, \dots, k, k, \dots, 2, 1$ if $k < N$, with $N - k + 1$ copies of k in the middle;
- $1, 2, \dots, N, N, \dots, 2, 1$ if $k \geq N$, with $k - N + 1$ copies of N in the middle.

Thus for every $k = 1, 2, \dots, T$, precompute the product of (coverage count + 1) for a column. This can be done using factorials and fast exponentiation.

It remains to maintain which columns are currently covered. Since the horizontal displacement can be large, direct range updates and clearing are too slow. For every currently covered column, store the time when its current covered interval began. A deque can maintain maximal consecutive column segments with the same start time. At each step, clear the segment leaving the rectangle and add the segment entering the rectangle with the current time. If the displacement has absolute value at least M , clear the entire deque. The time complexity is

$$O(T(\log T + \log N + \log M)),$$

with fast exponentiation being the bottleneck.

Problem G. Circular Parterre

The answer radius is at least the maximum input radius. Consider a candidate radius larger than that. Then the circular arcs on the boundary of the union of the input circles no longer give direct constraints; only the vertices of the union matter. The Voronoi diagram of these union vertices gives all possible candidate centers for the largest circle.

The algorithm is as follows.

1. Compute the union of the input circles. A polar-angle sort plus sweep-line method gives an $O(n^2 \log n)$ algorithm. Theoretically this step can be improved by using a power diagram.
2. Compute the vertices of the Voronoi diagram of the union vertices. Using properties of power diagrams, the number of union vertices is at most $6n$. This step can be implemented with half-plane intersection in $O(n^2 \log n)$ time, or by randomized incremental cutting of a convex polygon in expected $O(n^2)$ time.
3. For each candidate center, check whether it lies inside the region obtained after straightening the boundary arcs of the circle union. The necessity is immediate, and the sufficiency follows from the fact that no illegal candidate can pass this test. A standard winding-number test gives $O(n)$ time per candidate, and there are at most $18n$ candidates.

The resulting implementation runs in $O(n^2 \log n)$ time, with an $O(n^2)$ candidate-checking part. More advanced geometric data structures can reduce the theoretical complexity to $O(n \log n)$.

Problem H. Homeward Glance

We need the dimension of the solution space of matrices commuting with the given matrix. This dimension is unchanged if we extend the base field, so we may work over the algebraic closure $K = \overline{\mathbb{F}_p}$.

Over K , the characteristic polynomial splits. For every eigenvalue λ , let

$$V_\lambda = \bigcup_{t \geq 1} \ker((A - \lambda I)^t)$$

be its generalized eigenspace. If $AB = BA$, then V_λ is invariant under B , so we can compute the contribution of each eigenvalue independently and add the answers.

Assume for the moment that A has only one eigenvalue λ . Define

$$\lambda_i = \dim \ker((A - \lambda I)^i) - \dim \ker((A - \lambda I)^{i-1}).$$

The sequence $\lambda_1 \geq \lambda_2 \geq \dots$ is the conjugate partition of the Jordan block sizes. A standard calculation for Jordan blocks shows that the dimension of the commuting space is

$$\sum_i \lambda_i^2.$$

It remains to recover this information over $\mathbb{F}_p$ without explicitly entering the algebraic closure. Use an $O(n^3)$ randomized algorithm to compute the rational canonical form, obtaining polynomials

$$\cdots \mid p_2 \mid p_1.$$

One way is to choose a random vector v , find the minimum polynomial satisfying $p_1(A)v = 0$, then repeat in the quotient space to obtain $p_2, p_3, \dots$.

Each $p_i(x)$ describes the largest Jordan block associated with each eigenvalue. Repeated square-free factorization separates the block lengths. At level i of a generalized eigenspace, we obtain square-free polynomials

$$\cdots \mid q_{i2}(x) \mid q_{i1}(x).$$

Since these polynomials are square-free, their degrees reveal the corresponding values of λ_i for each eigenvalue. Summing $\sum_i \lambda_i^2$ over all eigenvalues gives the answer. The overall time complexity is $O(n^3)$.

Problem I. Deciding Game

Let c_A and c_B be the numbers of letters A and B in one period of the string, and let

$$\Delta = c_A - c_B.$$

For a time $t = qn + r$, where $0 \leq r < n$, the scores are

$$A(t) = qc_A + \text{pref}_A(r), \quad B(t) = qc_B + \text{pref}_B(r),$$

and the score difference is

$$D(t) = A(t) - B(t) = q\Delta + \text{pref}_D(r).$$

A wins exactly when

$$A(t) \geq m \quad \text{and} \quad D(t) \geq 2,$$

and B wins exactly when

$$B(t) \geq m \quad \text{and} \quad D(t) \leq -2.$$

For each residue r , these are just linear inequalities in the integer variable q . We can compute the smallest nonnegative q satisfying them, if it exists, and obtain the candidate time $qn + r$. Taking the minimum over all residues and over both players gives the answer.

Care is needed for zero coefficients. For example, if $c_A = 0$, then the inequality $A(t) \geq m$ is possible only if the prefix score already reaches m . Similarly, if $\Delta = 0$, the lead condition depends only on the prefix; if $\Delta < 0$, the condition $D(t) \geq 2$ gives an upper bound on q rather than a lower bound. The B-winning case is symmetric.

Thus each test case is solved by one scan over the n residues. If no candidate time exists, output “No”; otherwise output the minimum candidate time. The time complexity is $O(n)$ per test case.

Problem J. Divide the String

Apply two transformations.

1. Replace every 0 by -1 and compute prefix sums s_i . Then an interval $[l, r]$ has absolute majority 1 if and only if

$$s_r - s_{l-1} \geq 1.$$

The condition for majority 0 is symmetric.

2. Merge adjacent equal characters of the target string. After merging, finding k consecutive intervals whose absolute majority is 1 becomes equivalent to finding an interval with

$$s_r - s_{l-1} \geq k.$$

Again, the majority-0 case is symmetric.

Let f_i be the minimum possible right endpoint after processing the first i merged blocks. Suppose the merged target has m blocks. For an internal block $1 < i < m$, assume by symmetry that it requires majority 1. Then a position j can be the right endpoint of the i -th block if and only if

$$j > f_{i-1} \quad \text{and} \quad s_j - \min_{t=f_{i-1}}^{j-1} s_t \geq k.$$

The forward direction is immediate from the definition of f_{i-1} . Conversely, if the above condition holds, choose a position t with $f_{i-1} \leq t < j$ and s_t minimal. We can move the right endpoint of the previous block to t and make the current block end at j ; the inequality guarantees the required majority.

Therefore we can scan the string to compute f_1 , then repeatedly scan to compute $f_2, \dots, f_{m-1}$ using the minimum prefix sum condition above, and finally check whether n can serve as the endpoint of the last block. The total time complexity is linear.

Problem K. Master of Modular Arithmetic

The only impossible case is the following: all target values b_i are equal, and at least one a_i is not already equal to that common value.

Why is this impossible? Suppose all targets are w . Consider the last operation. It must multiply one position by some x so that it becomes w , hence x is a divisor of w . The paired modulo operation on the other position cannot produce w modulo such an x . Therefore the all-equal target cannot be reached unless it was already reached before the final operation.

It remains to show that every other case is constructible. Choose a prime $P > 3n$, and sort the positions by their target values b_i . First, use $O(1)$ operations to make $a_1 = 1$, sending the corresponding multiplication side effects to a_n .

For every middle position $i = 2, 3, \dots, n-1$, transform a_i directly into b_i in two main operations. Choose the smallest integer $k \geq 3$ such that $a_i k > 2b_i$. Multiply a_i by k , then use a modulo operation to obtain b_i . The two side effects, namely modulo by k and multiplication by $a_i k - b_i$, are assigned to a_1 and a_n . After processing all middle positions, we have $a_1 = 1$, while a_n is a large number not divisible by P .

Now work only with a_1 and a_n . Compute the inverse of a_n modulo P . If the inverse is not 1, multiply a_n by it and apply the paired modulo operation to a_1 . Then reduce a_n modulo P ; the pair becomes $(P, 1)$. One more operation with value 2 changes it to $(1, 2)$.

Finally, the pair $(1, 2)$ can be expanded into any pair (x, y) with $x \neq y$. For example, to obtain $(1, x)$, multiply directly if x is even; if x is odd, first get $(1, x+1)$ and then apply one operation with value x to obtain $(x, 1)$. Swapping the order can be done by a constant number of extra operations. This completes the construction.

Problem L. Not Another Constructive Problem

Relabel the permutation so that initially $p_i = i$. Thus every vertex starts as a one-element cycle. Because the given graph is a tree, whenever an edge is deleted in the forward process, its endpoints lie in different cycles, so the operation merges two cycles. Hence after all operations there must be exactly one cycle. If the target permutation q has more than one cycle, the answer is immediately invalid.

Now reverse the process. We split one cycle into two cycles. After a split, the two new cycles are independent. Therefore all tree edges must be noncrossing when drawn inside the cycle order. This condition is also sufficient.

To see sufficiency, choose an edge (x, y) . It splits the cycle into two arcs, say L and R . We need x to have no edge into R , and y to have no edge into L . Equivalently, if every vertex orders its incident edges along the cycle in the same direction, the chosen edge must be the first edge of both endpoints. Start from any vertex y and follow the first edge to a vertex x . If that edge is also the first edge of x , we can split there. Otherwise continue following first edges. Since the edges are noncrossing, this process eventually finds a valid edge. After splitting, induction applies to the two subcycles.

The problem is now to count spanning trees whose edges are noncrossing in a cycle order. View the cycle as a sequence. Once the connections around the endpoints of an interval are fixed, the remaining parts break into independent subintervals, so interval DP applies.

Let $f_{l,r,0/1}$ be the number of ways inside interval $[l, r]$, where the last bit indicates whether l and r are connected. A direct transition costs $O(n^4)$, and standard prefix-sum optimization reduces it to $O(n^3)$.

Problem M. Dividing Chains

Think of the current sequence b as being partitioned into segments. In one operation, choose a segment, cut it into two halves, and optionally swap the two halves. The important point is that the swap changes only the corresponding part of b ; the positions of a are fixed.

For any segment $[l, r]$, the multiset of $a_l, \dots, a_r$ must be equal to the multiset of $b_l, \dots, b_r$; otherwise this segment can never be transformed correctly.

For a legal segment $[l, r]$, define two sets:

- S_0 is the set of cuts x such that cutting at x without swapping keeps both child segments legal;
- S_1 is the set of cuts x such that cutting at x and swapping the two child segments keeps both child segments legal.

A key property is that all legal operations may be performed in arbitrary order.

In most cases, at least one of S_0 and S_1 is empty. If both are nonempty, then necessarily

$$b_l = b_r = a_l \quad \text{or} \quad b_l = b_r = a_r.$$

Define

$$\begin{aligned} dp_{l,r,0} &: \text{number of ways for segment } [l, r] \text{ when } S_1 \text{ is empty,} \\ dp_{l,r,1} &: \text{number of ways for segment } [l, r] \text{ when } S_0 \text{ is empty,} \\ dp_{l,r,2} &: \text{total number of ways for segment } [l, r]. \end{aligned}$$

Consider computing $dp_{l,r,0}$; the other case is symmetric. Let x be the minimum element of S_0 , and perform this cut first. Then the left child must have empty S_0 , while the right child is unrestricted, so the transition has the form

$$dp_{l,r,0} \leftarrow dp_{l,x,1} \cdot dp_{x+1,r,2}.$$

This counts cases where S_0 is nonempty, so we must subtract the cases where both S_0 and S_1 are nonempty.

It remains to count the overlapping case. Suppose $b_l = b_r = a_l$; the case $b_l = b_r = a_r$ is symmetric. If $a_l < a_{l+1}$, the number of such ways is 0. Otherwise $a_l = a_{l+1}$. Then $l \in S_0$ and $r-1 \in S_1$, so both sets are already nonempty. Since b_l and b_r are fixed to a_l , it remains only to transform

$$b_{l+1}, \dots, b_{r-1}$$

into

$$a_{l+2}, \dots, a_r,$$

which contributes $dp_{l+2,r,2}$.

Filling the interval DP in increasing length gives total time complexity $O(n^3)$.